UBER FARE PREDICTION

Complete Code Explanation • Machine Learning Theory • Practical Viva Questions

ASSIGNMENT TOPIC Regression Analysis	**MODELS** Linear • Ridge • Lasso
DATASET Uber ride records (uber.csv)	**MAIN TASK** Predict ride fare in USD

A beginner-friendly guide written directly around the code in your uploaded Jupyter notebook. It explains what the code does, why it is done, what each function means, and how to answer follow-up questions in a viva.

Viva strategy: First explain the problem, then the data preparation, then the distance calculation, then the three models, and finally how you compare their predictions.

Prepared from the uploaded Assignment_3(A).ipynb and the Computer Laboratory-I assignment list. The assignment list describes Uber fare prediction under Regression Analysis (Group A, item 3A).

1. Understand the project

Problem statement: Predict the price (fare amount) of an Uber ride using information about the trip. The notebook uses pickup and drop-off latitude/longitude, passenger count, pickup time, and a calculated trip distance.

What goes in and what comes out?

• Input/features (X): distance in kilometres, passenger count, pickup hour, and pickup year.

• Target (y): **fare_amount**, the fare in US dollars.

• This is supervised learning because historical examples contain both input values and the correct fare.

• It is a regression problem because the output is a continuous numerical value, not a class such as 'yes/no'.

Overall workflow

1. Load data	2. Inspect data	3. Clean records	4. Calculate distance
5. Create features	6. Split train/test	7. Scale features	8. Train + evaluate

2. Imports: what every library does

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

pandas (pd): Works with tabular data using DataFrames and Series. Used here to read CSV files, inspect columns, remove duplicates, and create features.

NumPy (np): Provides fast numerical arrays and mathematical functions. Used for radians, sine, cosine, square root, clipping and square root of MSE.

Matplotlib (plt): Used to draw the fare box plot and display it.

scikit-learn: Supplies machine-learning models, preprocessing, data splitting and evaluation metrics.

Note: make_pipeline is imported but not used in this notebook. Mention this honestly if asked. It can combine preprocessing and a model into one pipeline.

3. Loading, inspecting and cleaning the data

Cell 1 — Read the CSV

```
df = pd.read_csv("uber.csv")
df.head()
```

pd.read_csv() reads a comma-separated file and returns a pandas DataFrame. The result is stored in df. df.head() displays the first five rows by default, helping us check column names and sample values.

Cell 2 — Inspect the dataset

```
df.shape
df.describe()
df.info()
```

• df.shape returns (number_of_rows, number_of_columns). It does not modify the data.

• df.describe() gives numerical summary statistics such as count, mean, standard deviation, minimum, quartiles and maximum.

• df.info() prints column names, non-null counts, data types and memory usage.

• In a notebook, if several expressions are written in one cell, usually only the final expression is automatically displayed. Use print() or display() to show each result clearly.

Cell 3 — Find missing values

```
df.isna().sum()
```

isna() creates a True/False result for each cell: True means missing. sum() counts True values column-wise because True behaves like 1 and False like 0.

Cell 4 — Keep useful columns and clean records

```
columns = ['fare_amount', 'pickup_datetime', 'pickup_longitude',
           'pickup_latitude', 'dropoff_longitude',
           'dropoff_latitude', 'passenger_count']
rides = df[columns].drop_duplicates().copy()

rides['pickup_datetime'] = pd.to_datetime(
    rides['pickup_datetime'], errors='coerce', utc=True)
rides = rides.dropna()
rides = rides[(rides['fare_amount'] > 0) &
              rides['passenger_count'].between(1, 6)]

for col in ['pickup_longitude', 'dropoff_longitude']:
    rides = rides[rides[col].between(-75, -72)]
for col in ['pickup_latitude', 'dropoff_latitude']:
    rides = rides[rides[col].between(40, 42)]
print(len(rides))
```

Why select columns? The model only needs these seven raw columns. Selecting them makes the working table simpler.

drop_duplicates() removes repeated rows. **copy()** makes an independent working copy, reducing accidental changes to the original DataFrame.

pd.to_datetime() converts text into datetime values. errors='coerce' turns invalid date strings into NaT (missing datetime); utc=True interprets/converts timestamps to UTC.

dropna() removes rows with at least one missing value among the selected columns.

between(a,b) checks inclusive range membership. Fare must be positive; passenger count is restricted to 1–6. Longitude and latitude bounds are rough geographic filters intended to retain trips around New York City.

Cross-question: Why not simply fill every missing value with zero? Because zero may be a real or physically invalid value depending on the column, and can mislead the model. First understand why values are missing; then choose removal or suitable imputation.

4. Distance calculation: Haversine formula

The notebook has pickup and drop-off coordinates. It converts latitude/longitude into an approximate distance along the Earth's surface, rather than using a flat straight-line distance in degrees.

```
lat1, lon1, lat2, lon2 = np.radians(
    rides[['pickup_latitude', 'pickup_longitude',
           'dropoff_latitude', 'dropoff_longitude']].to_numpy()
).T

a = (np.sin((lat2-lat1)/2)**2
     + np.cos(lat1)*np.cos(lat2)
     * np.sin((lon2-lon1)/2)**2)

rides['km_dist'] = 6371 * 2 * np.arcsin(np.sqrt(np.clip(a, 0, 1)))
rides = rides[rides['km_dist'] > 0].copy()

rides['hour'] = rides['pickup_datetime'].dt.hour
rides['year'] = rides['pickup_datetime'].dt.year
features = ['km_dist', 'passenger_count', 'hour', 'year']
```

Line-by-line explanation

• to_numpy() converts the selected DataFrame columns into a NumPy array.

• np.radians() converts degrees to radians. Trigonometric functions in NumPy expect radians.

• .T transposes the array so the four columns can be unpacked into four arrays: latitude 1, longitude 1, latitude 2, longitude 2.

• np.sin() and np.cos() calculate sine and cosine element-wise.

• np.clip(a, 0, 1) restricts tiny floating-point deviations to the valid interval [0,1].

• np.sqrt() and np.arcsin() complete the Haversine calculation.

• 6371 is the approximate Earth's radius in kilometres. Multiplying by it gives distance in kilometres.

• .dt.hour extracts the hour (0–23); .dt.year extracts the year.

Haversine formula

For two points on a sphere: $a = \sin^2(\Delta\phi/2) + \cos(\phi1)\cos(\phi2)\sin^2(\Delta\lambda/2)$; distance $= 2R \times \text{asin}(\sqrt{a})$. Here ϕ is latitude in radians, λ is longitude in radians, and $R \approx 6371$ km.

Important distinction: This is geographic straight-line/ great-circle distance. It is not the actual road distance or travel distance. A real taxi route can be longer because roads curve, traffic rules apply, and bridges/roads constrain the route.

Q. Why use Haversine instead of subtracting latitude and longitude?
Answer: Latitude and longitude are angles on a sphere, not ordinary Cartesian distances. Haversine accounts for Earth's curvature and gives a much better surface-distance estimate.

Q. Why remove km_dist <= 0?
Answer: A zero distance may indicate identical pickup/drop-off coordinates or an invalid record. The notebook removes these because they are not useful for this fare model.

Q. What does vectorization mean here?
Answer: NumPy performs the calculation on whole arrays at once, rather than using a Python loop for every ride. This is generally faster and more concise.

5. Explore outliers, split data and scale features

Cell 6 — Visual inspection and correlation

```
rides.boxplot(column='fare_amount')
plt.title('Fare outliers before IQR filtering')
plt.ylabel('Fare(USD)')
plt.show()

display(rides[features + ['fare_amount']].corr())
```

Box plot: Shows the median, lower/upper quartiles, whiskers and points beyond the whiskers. It helps spot unusually low or high fare values. plt.title() adds a title, plt.ylabel() labels the vertical axis, and plt.show() renders the plot.

corr(): Calculates pairwise Pearson correlation by default for numeric columns. Values range from -1 to +1: positive means variables tend to increase together; negative means one tends to decrease as the other increases; near zero means weak linear association. Correlation does not prove causation.

Cell 7 — Train/test split and IQR filtering

```
train, test = train_test_split(
    rides, test_size=0.2, random_state=42)

for col in ['fare_amount', 'km_dist']:
    q1 = train[col].quantile(0.25)
    q3 = train[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    train = train[(train[col] >= lower) &
                  (train[col] <= upper)]
```

test_size=0.2 reserves 20% for testing and 80% for training. random_state=42 makes the random split reproducible. The IQR is Q3 – Q1; values below Q1 – 1.5×IQR or above Q3 + 1.5×IQR are flagged as outliers by this rule.

Important code observation: Outlier filtering is applied to train only; the test set is not filtered. This keeps the test set separate, but extreme test fares/distances can make evaluation harder. Do not claim that outliers were removed from the entire dataset.

```
X_train = train[features]
y_train = train['fare_amount']
X_test = test[features]
y_test = test['fare_amount']
```

X contains input features; y contains the value to predict. The model learns from X_train and y_train, then is evaluated on X_test against y_test.

Why scale?

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

StandardScaler standardizes each feature using training-set mean and standard deviation: z = (x – mean) / standard deviation. fit_transform() learns the mean/std from training data and transforms it. On test data, use only transform() so the test set does not influence preprocessing statistics.

Classic viva question: Why not call fit_transform on the test set too? That would calculate a different scale using test information, creating an inconsistent evaluation and data leakage. Fit preprocessing on training data only, then reuse it on test data.

6. The three regression models

All three models predict a numerical fare. The difference is how they estimate coefficients and whether they penalize large coefficients.

A. Linear Regression

Model idea: ■ = b0 + b1x1 + b2x2 + ... + bnxn. The model finds coefficients that minimize the sum of squared differences between actual and predicted fares.

In this notebook, features are km_dist, passenger_count, hour and year. The intercept b0 is the predicted baseline when standardized features are zero. Each coefficient describes the predicted fare change for a one-unit change in that standardized feature, holding the others fixed.

B. Ridge Regression (L2 regularization)

Ridge minimizes: Sum of squared errors + α × Sum of squared coefficients. It discourages large coefficients by penalizing their squares.

The notebook uses Ridge(alpha=1). Alpha controls regularization strength. Higher alpha generally shrinks coefficients more. Ridge usually keeps all features, but their weights become smaller. It can help reduce overfitting and handle correlated features.

C. Lasso Regression (L1 regularization)

Lasso minimizes: Sum of squared errors + α × Sum of absolute coefficient values.

The notebook uses Lasso(alpha=0.01). L1 regularization can drive some coefficients exactly to zero, so it can perform feature selection. If alpha is too high, it may underfit.

Code used to create and train models

```
linear = LinearRegression()
ridge = Ridge(alpha=1)
lasso = Lasso(alpha=0.01)

linear.fit(X_train_scaled, y_train)
linear_pred = linear.predict(X_test_scaled)

ridge.fit(X_train_scaled, y_train)
ridge_pred = ridge.predict(X_test_scaled)

lasso.fit(X_train_scaled, y_train)
lasso_pred = lasso.predict(X_test_scaled)
```

fit(X,y) trains/estimates the model parameters using examples and known targets. predict(X) uses the trained model to generate predictions for new feature rows.

Q. Why scale before Ridge and Lasso?
Answer: Their penalty depends on coefficient sizes. If one feature is measured in huge units and another in small units, the penalty can treat them unfairly. Scaling puts features on a comparable scale.

Q. Does Linear Regression require scaling?
Answer: For ordinary least squares, scaling is usually not required for the mathematical prediction quality, although it can help numerical stability and coefficient comparability. It is particularly important for regularized models.

7. Model evaluation: R² and RMSE

```
linear_r2 = r2_score(y_test, linear_pred)
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_pred))

ridge_r2 = r2_score(y_test, ridge_pred)
ridge_rmse = np.sqrt(mean_squared_error(y_test, ridge_pred))

lasso_r2 = r2_score(y_test, lasso_pred)
lasso_rmse = np.sqrt(mean_squared_error(y_test, lasso_pred))

results = pd.DataFrame({
    'Model': ['Linear', 'Ridge', 'Lasso'],
    'R2': [linear_r2, ridge_r2, lasso_r2],
    'RMSE': [linear_rmse, ridge_rmse, lasso_rmse]
})
display(results)
```

R² — coefficient of determination

R² compares the model's squared prediction errors with the error from predicting the test-set mean for every example. A value closer to 1 is generally better. R² = 1 means perfect predictions; R² = 0 means no improvement over predicting the mean; it can be negative when the model performs worse than that baseline.

RMSE — root mean squared error

RMSE = √[(1/n) Σ(actual − predicted)²]

RMSE measures the typical size of prediction error with larger errors penalized more heavily because errors are squared. Since the target is USD, RMSE is also in USD. Lower RMSE is generally better.

How to decide the best model?

• Prefer higher R² and lower RMSE, measured on the same test set.

• If one model has higher R² but also higher RMSE, examine the trade-off and the actual values.

• Do not invent a winning model: the notebook creates a results table, but the notebook file itself does not contain saved output values. Run it to see the actual scores.

• A model with good test performance should also be checked for generalization, data quality and realistic inputs.

Q. Why take square root of MSE?
Answer: MSE is in squared dollars. Taking the square root returns the error to the original dollar unit, making it easier to interpret.

Q. Can R² be negative?
Answer: Yes. It means the model's predictions are worse, in squared-error terms, than simply predicting the mean fare for all test examples.

Q. Is accuracy suitable for this task?
Answer: No. Accuracy is mainly for classification. Fare prediction is regression, so metrics such as MAE, MSE, RMSE and R² are more suitable.

Useful extra metric: MAE = average absolute error. It is measured in USD and is less sensitive to very large errors than RMSE.

8. Theory viva: core machine-learning concepts

Q. What is Machine Learning?
Answer: A field where computers learn patterns from data to make predictions or decisions without writing a separate rule for every case.

Q. What is supervised learning?
Answer: Learning from labelled examples. Here each ride has input information and its known fare, so the model learns the mapping from trip features to fare.

Q. Regression vs classification?
Answer: Regression predicts a continuous number, such as fare or temperature. Classification predicts a category, such as spam/not spam or digit 0–9.

Q. What are features and target?
Answer: Features are the input columns used to make a prediction. The target is the value the model is trying to predict. Here features are distance, passenger count, hour and year; target is fare_amount.

Q. What is overfitting?
Answer: When a model learns training data too closely, including noise, and performs poorly on unseen data.

Q. What is underfitting?
Answer: When a model is too simple or too constrained to capture the pattern, so it performs poorly even on training data.

Q. Bias-variance trade-off?
Answer: High bias often means an overly simple model; high variance means the model is too sensitive to the training sample. The aim is to generalize well to unseen rides.

Q. What is regularization?
Answer: A technique that adds a penalty for large model coefficients to discourage overly complex fits. Ridge uses L2; Lasso uses L1.

Q. What is data leakage?
Answer: When information from the test set or future data influences training or preprocessing. It makes evaluation look better than real-world performance.

Q. Why keep a test set?
Answer: It provides an estimate of how well the trained model predicts examples it did not learn from.

Q. What is random_state?
Answer: A seed used by a random process. Keeping it fixed makes the split repeatable, so results can be compared consistently.

Q. What is multicollinearity?
Answer: When input features are strongly correlated with one another. It can make ordinary linear regression coefficients unstable; Ridge can reduce this instability.

Q. What assumptions are associated with classical linear regression?
Answer: A linear relationship in parameters, independent errors, constant error variance (homoscedasticity), and no perfect multicollinearity. Normal residuals are mainly relevant for some statistical inference tests.

Q. What is an outlier?
Answer: An observation unusually far from the general pattern. It may be an error or a valid rare event; investigate before removing it.

Q. What is the IQR method?
Answer: IQR = Q3 − Q1. The common rule flags values below Q1 − 1.5×IQR or above Q3 + 1.5×IQR.

9. Code cross-questions, limitations and improvements

Functions and parameters examiners may ask

Q. What does DataFrame.copy() do?
Answer: Creates a separate copy of the selected data. It helps avoid unintentionally modifying the original DataFrame through a shared view.

Q. What is errors='coerce' in to_datetime?
Answer: Invalid date strings become NaT instead of raising an error, allowing them to be removed or handled.

Q. What does utc=True do?
Answer: Returns timezone-aware datetime values normalized to UTC. Be careful: extracting hour from UTC may not represent local pickup hour.

Q. What does quantile(0.25) return?
Answer: The first quartile (25th percentile); quantile(0.75) returns the third quartile (75th percentile).

Q. What is alpha in Ridge/Lasso?
Answer: The regularization strength. Larger alpha means a stronger penalty. It should ideally be selected using validation or cross-validation.

Q. What is fit_transform versus transform?
Answer: fit_transform learns preprocessing parameters from data and applies the transformation. transform applies already learned parameters without refitting.

Q. What does np.clip(a,0,1) protect against?
Answer: Floating-point rounding can make a theoretically valid Haversine intermediate slightly less than 0 or greater than 1. Clipping keeps arcsin's input valid.

Q. Why use .dt accessor?
Answer: It exposes datetime components such as hour, day, month and year from a pandas datetime Series.

Limitations of this exact notebook

• **UTC hour:** The code uses utc=True and then extracts hour. If the intended time feature is New York local time, convert to the correct timezone before extracting hour.

• **Distance is not road distance:** Haversine estimates great-circle distance and does not include road routes or traffic.

• **Only four features:** Fare can also depend on traffic, route, tolls, surge pricing, time of week, weather, airport fees and service rules.

• **Outlier handling:** Filtering is performed on training data only, while test examples remain unchanged. Report this accurately and consider robust evaluation.

• **Random split:** A random split may mix earlier and later rides. For forecasting future rides, a chronological split is often more realistic.

• **No hyperparameter tuning:** alpha values are fixed at 1 and 0.01. Cross-validation can help choose them.

• **Unused import:** make_pipeline is imported but not used.

Improvements you can confidently suggest

• Convert timestamps to the correct local timezone before extracting hour; add day-of-week and month.

• Use a Pipeline combining StandardScaler and the regression model to make preprocessing safer and reproducible.

• Use cross-validation and GridSearchCV to tune Ridge/Lasso alpha.

• Evaluate MAE as well as RMSE and R^2; inspect residual plots.

• Use a time-based train/test split if the goal is predicting future rides.

• Consider nonlinear models or interaction features if fare patterns are not adequately linear.

One-minute viva summary: "I built a supervised regression model to predict Uber ride fare. I loaded and inspected the CSV using pandas, removed duplicate and invalid records, converted pickup time to datetime, and filtered coordinates to the New York region. I calculated pickup-to-drop-off great-circle distance using the Haversine formula, then extracted hour and year. I used distance, passenger count, hour and year as features. I split the data into training and testing sets, removed training outliers using the IQR rule, standardized features using StandardScaler, and trained Linear, Ridge and

Lasso regression models. Finally, I compared them using R^2 and RMSE."

Source alignment: The uploaded lab assignment list, page 2, item 3A, specifies Uber ride fare prediction, preprocessing, outlier identification, correlation, Linear/Ridge/Lasso models and comparison using metrics such as R^2 and RMSE. This guide follows the uploaded notebook's actual implementation.